

- [RAG Studio — Framework Documentation](#)
  - [Table of contents](#)
  - [1. Architecture at a glance](#)
  - [2. Background you need](#)
    - [2.1 Embeddings and cosine similarity](#)
    - [2.2 Approximate Nearest Neighbour \(ANN\) — HNSW](#)
    - [2.3 Lexical retrieval — TF-IDF / BM25 / Postgres FTS](#)
    - [2.4 Reciprocal Rank Fusion \(RRF\)](#)
    - [2.5 Maximum Marginal Relevance \(MMR\)](#)
  - [3. Ingestion pipeline](#)
    - [3.1 Text extraction + chunking](#)
    - [3.2 Embedding generation](#)
    - [3.3 Anthropic Contextual Retrieval \(Sept 2024\)](#)
    - [3.4 Storage layout — pgvector + tsvector + S3](#)
  - [4. Retrieval pipeline](#)
    - [4.1 Query rewriting \(multi-query retrieval\)](#)
    - [4.2 HyDE — Hypothetical Document Embeddings](#)
    - [4.3 Query embedding](#)
    - [4.4 Dense retrieval \(HNSW\)](#)
    - [4.5 Sparse retrieval \(Postgres FTS\)](#)
    - [4.6 Hybrid fusion \(RRF\)](#)
    - [4.7 LLM listwise reranking \(RankGPT / RankZephyr\)](#)
    - [4.8 MMR diversification](#)
    - [4.9 CRAG self-grading](#)
  - [5. Generation pipeline](#)
    - [5.1 Prompt structure and citation grounding](#)
    - [5.2 Streaming SSE + token accounting](#)
  - [6. Deployment topology](#)
    - [6.1 Provider abstraction — Stellar vs Vertex](#)
    - [6.2 Remote ingestion service](#)
    - [6.3 S3 / MinIO object store](#)
  - [7. End-to-end algorithmic walkthrough](#)
  - [8. Tuning cheatsheet](#)
  - [9. References](#)
    - [Foundational papers](#)
    - [Modern RAG techniques](#)
    - [Storage / infrastructure](#)

# RAG Studio — Framework Documentation

---

A complete walkthrough of the retrieval pipeline, the strategies it implements, the algorithms behind each step, and the theory you need to read the code.

---

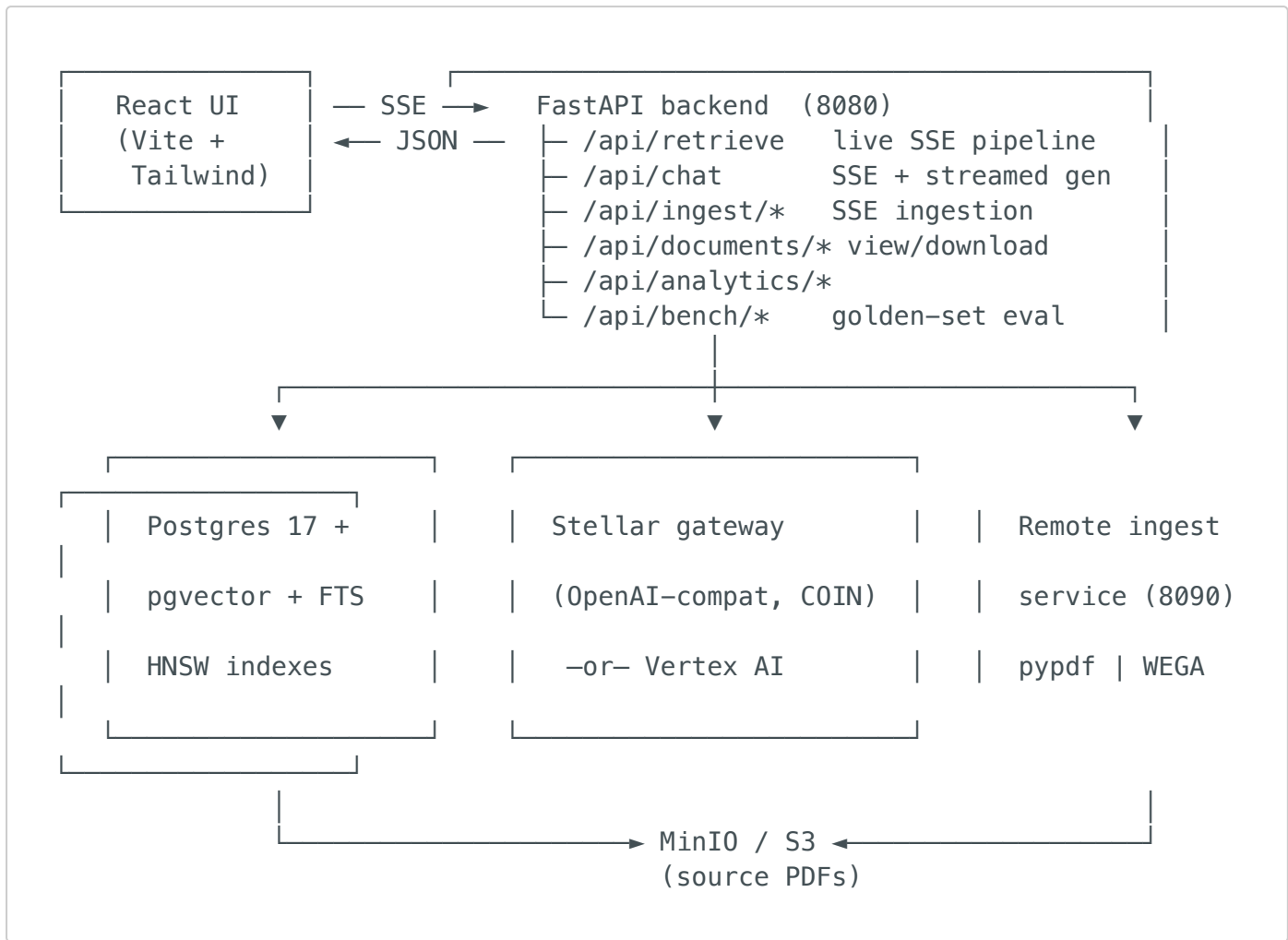
## Table of contents

---

1. [Architecture at a glance](#)
2. [Background you need](#)
  - [2.1 Embeddings and cosine similarity](#)
  - [2.2 Approximate Nearest Neighbour \(ANN\) — HNSW](#)
  - [2.3 Lexical retrieval — TF-IDF / BM25 / Postgres FTS](#)
  - [2.4 Reciprocal Rank Fusion \(RRF\)](#)
  - [2.5 Maximum Marginal Relevance \(MMR\)](#)
3. [Ingestion pipeline](#)
  - [3.1 Text extraction + chunking](#)
  - [3.2 Embedding generation](#)
  - [3.3 Anthropic Contextual Retrieval](#)
  - [3.4 Storage layout — pgvector + tsvector + S3](#)
4. [Retrieval pipeline](#)
  - [4.1 Query rewriting](#)
  - [4.2 HyDE — Hypothetical Document Embeddings](#)
  - [4.3 Query embedding](#)
  - [4.4 Dense retrieval](#)
  - [4.5 Sparse retrieval](#)
  - [4.6 Hybrid fusion \(RRF\)](#)
  - [4.7 LLM listwise reranking \(RankGPT\)](#)
  - [4.8 MMR diversification](#)
  - [4.9 CRAG self-grading](#)
5. [Generation pipeline](#)
  - [5.1 Prompt structure and citation grounding](#)
  - [5.2 Streaming SSE + token accounting](#)
6. [Deployment topology](#)
  - [6.1 Provider abstraction — Stellar vs Vertex](#)

- 6.2 [Remote ingestion service](#)
  - 6.3 [S3 / MinIO object store](#)
7. [End-to-end algorithmic walkthrough](#)
  8. [Tuning cheatsheet](#)
  9. [References](#)

# 1. Architecture at a glance



The system implements four orthogonal capabilities that can be toggled independently at retrieval time:

| Capability          | Default   | Where it sits in the pipeline      |
|---------------------|-----------|------------------------------------|
| Query rewriting     | off       | Pre-embedding (multi-query expand) |
| HyDE                | off       | Pre-embedding (hypothetical doc)   |
| Contextual chunking | <b>on</b> | Ingestion (Anthropic prefix)       |
| Hybrid search       | <b>on</b> | Dense + sparse parallel + RRF      |

| Capability          | Default   | Where it sits in the pipeline |
|---------------------|-----------|-------------------------------|
| LLM listwise rerank | <b>on</b> | Post-fusion                   |
| MMR                 | <b>on</b> | Final top-K selection         |
| CRAG self-grading   | off       | Post-rerank diagnostic        |
| Answer generation   | <b>on</b> | Final stage                   |

## 2. Background you need

### 2.1 Embeddings and cosine similarity

An **embedding** is a fixed-length vector that represents a piece of text in a high-dimensional space. Texts with similar meaning land near each other.

We use **gte-large-en-v1.5** (1024-dim) in Stellar and **text-embedding-005** (768-dim) in Vertex. The dim is discovered at runtime so the schema adapts.

**Cosine similarity** measures the angle between two vectors:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|}$$

For normalised vectors, this collapses to a dot product. pgvector exposes three operators: **<->** (L2), **<=>** (cosine **distance** = 1 – cos), and **<#>** (negative dot product). We use **<=>** because the embedding models are L2-normalised.

In SQL:

```
SELECT id, 1 - (embedding <=> $1::vector) AS score
FROM   vector.chunk_embeddings
ORDER BY embedding <=> $1::vector
LIMIT  50;
```

## 2.2 Approximate Nearest Neighbour (ANN) — HNSW

Exact nearest-neighbour search is  $O(n \cdot d)$ . For millions of chunks this is unacceptable.

**HNSW** (Hierarchical Navigable Small World, Malkov & Yashunin 2016) builds a multi-layer graph where the top layer has long-range links and lower layers densify down to the data.

**Search complexity:**  $O(\log n)$  expected. **Recall:** 95–99% at default parameters ( $m=16$ ,  $ef\_construction=64$ ). **Trade-off:** indexing cost + RAM for graph nodes.

pgvector exposes:

```
CREATE INDEX ON vector.chunk_embeddings  
USING hnsw (embedding vector_cosine_ops);
```

`vector_cosine_ops` tells HNSW to optimise for cosine distance.

## 2.3 Lexical retrieval — TF-IDF / BM25 / Postgres FTS

Embeddings miss exact-token matches (ticker symbols, product codes, named entities). Lexical search counters that.

**TF-IDF** weighs each term by how rare it is across the corpus:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log(N / \text{df}(t))$$

**BM25** (Robertson 1995) adds length normalisation and saturation so that the 10th occurrence of a term in one doc doesn't dominate:

$$\text{bm25}(t, d) = \text{idf}(t) \cdot \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1 \cdot (1 - b + b \cdot |d| / \text{avgdl})}$$

Default  $k_1 = 1.2$ ,  $b = 0.75$ .

**Postgres FTS** uses `ts_rank_cd`, an approximation of BM25 over a `tsvector` column. We use `websearch_to_tsquery` (Postgres 11+) which accepts Google-style queries ("`phrase`" `-negate`) without raising on punctuation:

```
SELECT id, ts_rank_cd(content_tsv, q) AS score
FROM   vector.chunk_embeddings,
       websearch_to_tsquery('english', $1) q
WHERE  content_tsv @@ q AND deleted_at IS NULL
ORDER  BY score DESC
LIMIT 50;
```

`content_tsv` is a **generated stored** column:

```
content_tsv tsvector GENERATED ALWAYS AS
(to_tsvector('english', COALESCE(content, ''))) STORED
```

A GIN index on `content_tsv` makes lookups sub-millisecond on hundreds of thousands of chunks.

## 2.4 Reciprocal Rank Fusion (RRF)

Cormack, Clarke, Büttcher 2009. Given multiple ranked lists  $L_1 \dots L_m$  over the same candidate set, assign each candidate a score:

$$\text{RRF}(c) = \sum_i \frac{1}{k + \text{rank}_i(c)}$$

where `rank_i(c)` is  $c$ 's 1-indexed position in list  $L_i$ , and  $k$  is a smoothing constant (we use  $k=60$  — the value from the original paper).

**Why it works:** the constant  $k$  damps the top — moving from rank 1 to 2 is worth far more than moving from 50 to 51. Documents that appear in multiple lists are rewarded even when they're not first in any single list. RRF needs **no calibration** — that's its killer feature compared to score normalisation schemes (min-max, z-score, log-rank) which fail when score distributions differ between retrievers.

Algorithm:

```

function RRF(rankings: dict[name → list[(id, payload)]], k = 60) →
list[hit]:
    scores ← {}
    payload ← {}
    for source_name, ranking in rankings.items():
        for rank, (id, p) in enumerate(ranking, start=1):
            scores[id] += 1 / (k + rank)
            payload[id] = payload.get(id) | p          # last writer wins
    return sorted(scores.items(), key=lambda x: -x[1])

```

## 2.5 Maximum Marginal Relevance (MMR)

Carbonell & Goldstein 1998. After retrieval you often have several near-duplicate chunks (the same passage with slightly different chunk boundaries). MMR greedily picks chunks that are simultaneously **relevant** to the query and **dissimilar** from chunks already chosen:

$$\text{MMR}(c \mid S) = \lambda \cdot \text{rel}(c, q) - (1 - \lambda) \cdot \max_{\{s \in S\}} \text{sim}(c, s)$$

- $S$  = already-selected set
- $\lambda \in [0, 1]$  —  $\lambda=1$  is pure relevance,  $\lambda=0$  is pure diversity
- We default to  $\lambda = 0.7$  (relevance-leaning).

Algorithm:

```

function MMR(candidates: list[(id, relevance, embedding)], λ, k) → list[id]:
    selected ← []
    pool ← candidates
    while len(selected) < k and pool:
        best_id, best_score ← None, -∞
        for c in pool:
            div = max(cos(c.emb, s.emb) for s in selected) if selected else
0            0
            score = λ · c.relevance - (1 - λ) · div
            if score > best_score: best_id, best_score = c, score
        selected.append(best_id)
        pool.remove(best_id)
    return selected

```

If embeddings aren't available the implementation falls back to a cheap diversity signal (different document / different page).

## 3. Ingestion pipeline

### 3.1 Text extraction + chunking

Two paths depending on environment:

| Mode               | Extractor       | Chunker                                            |
|--------------------|-----------------|----------------------------------------------------|
| Stellar (VDI prod) | Azure DI (WEGA) | WEGA Chunker SDK — semantic, PII-aware, 500 toks   |
| Vertex (local)     | <b>pypdf</b>    | Paragraph-aware char chunker (1500 chars / 150 OL) |

The local-Vertex chunker algorithm (see [backend/app/pipeline/local\\_ingest.py](#)):

```
function chunk_text(text, max_chars=1500, overlap=150):
    paragraphs ← text.split("\n\n").strip()
    chunks, cur ← [], ""
    for p in paragraphs:
        if len(cur) + len(p) + 2 ≤ max_chars:
            cur ← cur ⊕ "\n\n" ⊕ p
        else:
            if cur: chunks.append(cur)
            if len(p) > max_chars:
                step ← max_chars - overlap
                for i in range(0, len(p), step):
                    chunks.append(p[i : i + max_chars])
            cur ← ""
        else:
            cur ← p
    if cur: chunks.append(cur)
    return chunks
```

The paragraph break preserves topical boundaries — far better than blindly slicing on character count.

### 3.2 Embedding generation

For each chunk we ask the active provider for its embedding. Embeddings are batched (default batch=16) to amortise network round-trips.

```

async function embed_corpus(chunks):
    for batch in batched(chunks, B=16):
        vectors ← await provider.embed([c.content for c in batch])
        for c, v in zip(batch, vectors):
            c.embedding ← v

```

The provider is selected by `LLM_PROVIDER` and resolved through a small strategy class so the rest of the pipeline is provider-agnostic.

## 3.3 Anthropic Contextual Retrieval (Sept 2024)

**The headline technique.** A chunk like *"Revenue grew 32%"* is ambiguous — whose revenue? The Anthropic paper proposes prepending a 50–100 token **LLM-generated prefix** that locates the chunk in the document:

*"This passage is from Microsoft's Q2 FY26 10-Q discussing Intelligent Cloud segment performance." → "Revenue grew 32% ..."*

The combined text is then embedded. Reported result: ~49% reduction in retrieval failures versus naive chunking.

Algorithm:

```

function contextualise(document, chunks):
    # Prompt the LLM once per chunk with full doc + the chunk
    for chunk in chunks:
        prompt = f"""
            Document: {document}
            Chunk: {chunk.content}
            Give a 50–100 token prefix that situates this chunk in the
            document. Return only the prefix.
        """
        prefix = await llm.chat(model=CONTEXT_MODEL, messages=[prompt])
        combined = prefix + "\n\n" + chunk.content
        chunk.context_embedding = await llm.embed_one(combined)
        chunk.context_text = prefix

```

We store **both** embeddings (raw and contextual) so retrieval can pick between them via a toggle. Cost is dominated by the LLM call per chunk — in our pipeline we use the **fast Stellar Llama-3.1-8B / Gemini-2.5-Flash** because the task is short and structured.

## 3.4 Storage layout — pgvector + tsvector + S3

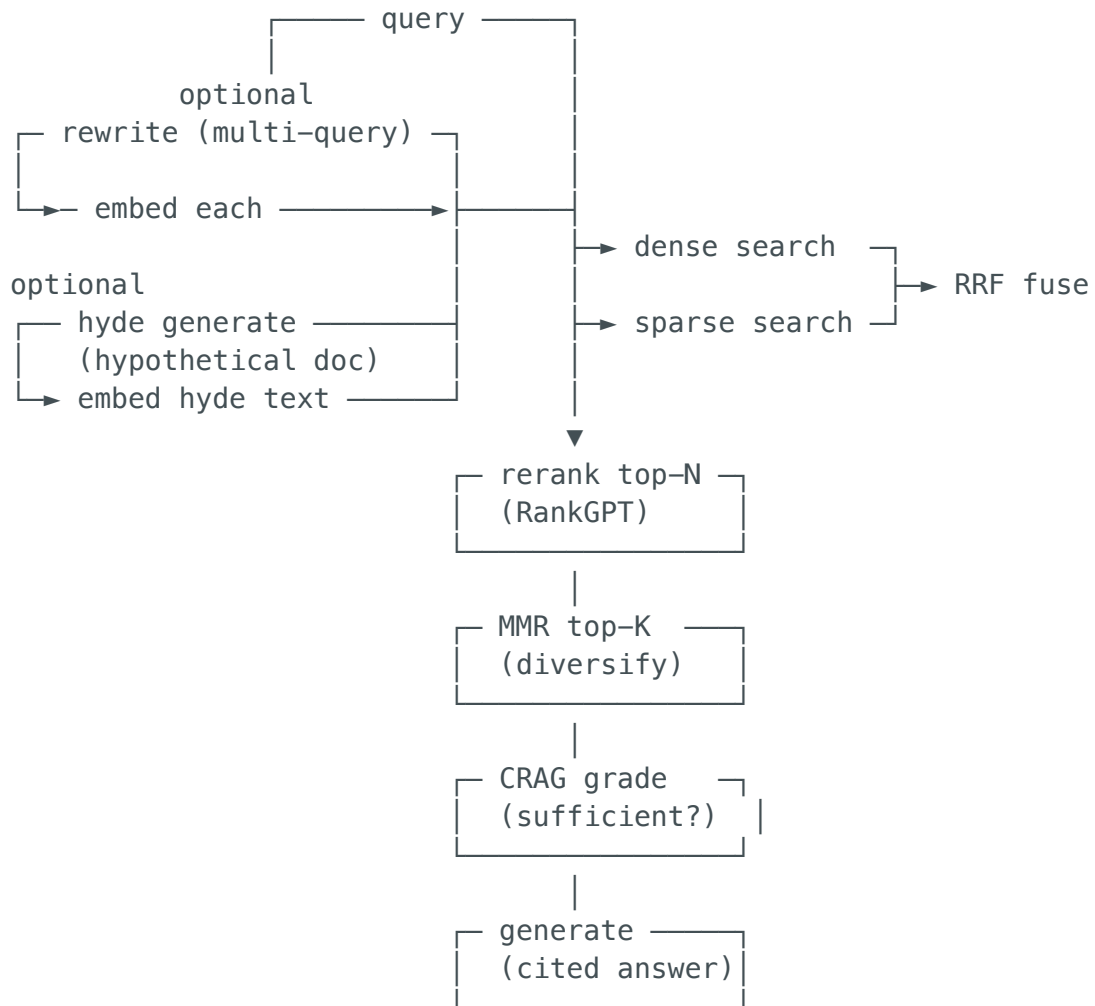
```
vector.chunk_embeddings (  
  id                BIGSERIAL PRIMARY KEY,  
  content           TEXT          NOT NULL,  
  "chunkUUID"       TEXT          UNIQUE,  
  "pageNumber"      INTEGER,  
  "tokenCount"      INTEGER,  
  "chunkType"       TEXT,  
  "documentName"    TEXT,  
  "jobId"           TEXT,  
  embedding         vector(D),  
  content_tsv       tsvector  
                    GENERATED ALWAYS AS (to_tsvector('english',  
                    COALESCE(content, ''))) STORED,  
  deleted_at        TIMESTAMPTZ  
);  
  
CREATE INDEX idx_chunk_embedding_hnsw ON vector.chunk_embeddings  
  USING hnsw (embedding vector_cosine_ops);  
CREATE INDEX idx_chunk_content_tsv ON vector.chunk_embeddings  
  USING gin (content_tsv);  
CREATE INDEX idx_chunk_active ON vector.chunk_embeddings  
  USING btree ("documentName") WHERE deleted_at IS NULL;  
  
vector.chunk_context (  
  chunk_id          BIGINT REFERENCES chunk_embeddings(id) ON DELETE  
  CASCADE,  
  context_text      TEXT,  
  context_embedding vector(D)  
);  
CREATE INDEX idx_chunk_context_embedding_hnsw ON vector.chunk_context  
  USING hnsw (context_embedding vector_cosine_ops);
```

- **Soft delete:** `UPDATE ... SET deleted_at = now()` — fast, reversible, and the partial index `WHERE deleted_at IS NULL` keeps queries the same as a hard delete would.
- **Source PDFs:** stored in MinIO/S3 at `s3://${bucket}/docs/<short_uuid>/<filename>`. A `documents` row carries the `s3_uri` so View/Download endpoints can stream the original back.

---

## 4. Retrieval pipeline

---



## 4.1 Query rewriting (multi-query retrieval)

Goal: catch vocabulary mismatch between the question and the documents.

```

function rewrite(query, n=3) → list[str]:
    prompt = "Rewrite the question in {n} different ways: a broader
              version, a more specific version, a keyword-oriented version.
              Return JSON array."
    raw = await llm.chat(model=FAST_MODEL, prompt=prompt.format(n=n,
q=query))
    variants = parse_json_array(raw)
    return [query] + variants          # original is always included
  
```

Each variant is embedded and feeds its own dense search; the results are unioned at the RRF stage. This is **multi-query retrieval** — a 2018 technique that pre-dates LLMs but is now trivial with one.

## 4.2 HyDE — Hypothetical Document Embeddings

Gao et al. 2022 ([arXiv:2212.10496](https://arxiv.org/abs/2212.10496)).

Vocabulary in *questions* is often very different from vocabulary in *answers*. We ask the LLM to draft a confident fake passage and embed that instead:

```
function hyde(query) → str:
    return await llm.chat(model=FAST_MODEL, messages=[{
        "role": "user",
        "content": f"Write a short, factual passage that would answer the
                    following question. Three to five sentences, no hedging.
                    Question: {query}"
    }])
```

The fake passage shares structural and vocabulary patterns with real answers in your corpus, lifting recall on under-specified queries. Critically, HyDE does **not** ask the LLM for facts — it just shapes the search query.

## 4.3 Query embedding

If either rewrite or hyde is on, multiple texts are embedded in parallel. Otherwise just the original query.

```
texts = [original]
if rewrite: texts += rewrites
if hyde:    texts += [hyde_passage]
vectors = await provider.embed(texts)    # batched
```

## 4.4 Dense retrieval (HNSW)

Per query vector, fetch the top **rag\_topk\_dense** (default 50) by cosine similarity. When **use\_contextual=True**, we hit the parallel index on **chunk\_context.context\_embedding**.

```
WITH q AS (SELECT $1::vector AS v)
SELECT c.id, c.content, c."documentName", c."pageNumber",
```

```

        ctx.context_text,
        1 - (c.embedding <=> q.v) AS score
FROM    vector.chunk_embeddings c
LEFT JOIN vector.chunk_context ctx ON ctx.chunk_id = c.id, q
WHERE   c.deleted_at IS NULL
ORDER BY CASE WHEN $use_contextual
              THEN ctx.context_embedding <=> q.v
              ELSE c.embedding              <=> q.v
            END
LIMIT   $top_k;

```

## 4.5 Sparse retrieval (Postgres FTS)

```

WITH q AS (SELECT websearch_to_tsquery('english', $1) AS tsq)
SELECT id, content, "documentName", "pageNumber",
       ts_rank_cd(content_tsv, q.tsq) AS score
FROM    vector.chunk_embeddings, q
WHERE   content_tsv @@ q.tsq AND deleted_at IS NULL
ORDER BY score DESC
LIMIT   $top_k;

```

`ts_rank_cd` is a Postgres approximation of cover density that behaves similar to BM25 but doesn't require building a separate index pipeline.

## 4.6 Hybrid fusion (RRF)

Each retriever's output is fed in as a ranked list keyed by its name; RRF returns a fused ranking. Source attribution is kept in `components` so the UI can show a per-source score breakdown.

```

fused = reciprocal_rank_fusion(
  rankings = {
    "dense_v0": dense_hits,
    "dense_v1": dense_hits_for_rewrite_1,
    "dense_hyde": dense_hits_for_hyde,
    "sparse": sparse_hits,
  },
  k = 60,
  top_k = max(rag_rerank_topn, rag_final_k),
)

```

After RRF we have a unified ranked list ready for reranking.

## 4.7 LLM listwise reranking (RankGPT / RankZephyr)

Sun et al. 2023 ([arXiv:2304.09542](https://arxiv.org/abs/2304.09542)).

Traditional rerankers use a cross-encoder (e.g. BGE-reranker, Cohere reranker). RankGPT instead presents the entire candidate list to a chat model and asks for a **permutation**:

Prompt:

You will be given a query and a numbered list of passages.  
Re-rank them from most relevant to least relevant for the query.  
Return ONLY a JSON array of the integer IDs, in your preferred order.

Query: {q}

Passages:

[1] (chars 0-400 of passage 1) ...  
[2] (chars 0-400 of passage 2) ...  
...  
[N] (chars 0-400 of passage N) ...

Model output:

[4, 1, 7, 2, 9, ...]

Algorithm:

```
async function llm_rerank(query, candidates: list[hit], N) → list[hit]:
    prompt = build_listwise_prompt(query, candidates[:N])
    raw = await llm.chat(model=RERANK_MODEL, prompt=prompt,
                        max_tokens=512, temperature=0)
    order = parse_json_array(raw)
    reordered = []
    for idx in order:
        if 1 ≤ idx ≤ N:
            reordered.append(candidates[idx - 1])
    # append anything the model dropped, preserving original order
    seen = set(id(c) for c in reordered)
    for c in candidates[:N]:
        if id(c) not in seen: reordered.append(c)
    return reordered
```

Defaults: top **N=20** candidates reranked. We use the **70B model** here ([Meta-Llama-3.3-70B-Instruct](#) on Stellar, [gemini-2.5-flash](#) on Vertex) because reranking quality dominates downstream answer quality.

**Cost note:** one rerank call typically consumes 3–5k prompt tokens and returns ~100 completion tokens. The Token Breakdown card in the UI shows the live spend per stage.

## 4.8 MMR diversification

After reranking we still want the final K (default 8) to **cover** the question rather than restate the same fact 8 times. MMR re-selects K items from the reranked top-N, optimising the joint relevance/diversity objective of §2.5.

```
selected = mmr_select(
    candidates = reranked[:rag_rerank_topn],
    λ           = settings.rag_mmr_lambda,      # 0.7
    k           = strategy.top_k,               # default 8
)
```

When embeddings are absent (e.g., MMR running on RRF output without a dense pass), the implementation falls back to a "different doc / different page" diversity signal — cheap but surprisingly effective.

## 4.9 CRAG self-grading

Yan et al. 2024 ([arXiv:2401.15884](#)).

After retrieval we can ask the model whether the retrieved context is actually enough to answer the question. The classifier returns one of {**correct**, **ambiguous**, **incorrect**} along with a confidence in [0, 1]:

```
Prompt:
Question: {q}
Retrieved passages: {top-K context}
Is the retrieved context sufficient to answer the question?
Return JSON: {"verdict": "correct"|"ambiguous"|"incorrect",
               "confidence": <float 0-1>, "reason": "<one line>"}
```

Action policy:

| Verdict        | What we do |
|----------------|------------|
| <b>correct</b> | proceed    |

| Verdict   | What we do                                                 |
|-----------|------------------------------------------------------------|
| ambiguous | proceed, but expose the badge so the user knows            |
| incorrect | proceed but flag — future enhancement: web-search fallback |

We deliberately use the **fast 8B model** here — it's a cheap second opinion that complements the rerank+generate pair.

## 5. Generation pipeline

### 5.1 Prompt structure and citation grounding

The generation prompt looks like:

```
You are a careful research assistant. Answer the user's question using
ONLY the provided context. Cite every factual claim with [N] markers
referencing the numbered context.
```

```
Context:
```

```
[1] (doc=10-Q_microsoft.pdf page=33 chunk #232)
    Details Microsoft Cloud revenue for three and six months ended ...
```

```
[2] (doc=10-Q_microsoft.pdf page=34 chunk #237)
    Microsoft Cloud revenue increased 26% to $51.5 billion ...
```

```
...
```

```
If the context doesn't answer the question, say so explicitly. End with
a "## Sources" section listing every [N] you cited with its doc name,
page, and chunk id.
```

```
Question: {q}
```

```
Answer:
```

The frontend post-processes the streamed text: every **[N]** token is substituted with a clickable chip that scrolls to (and briefly highlights) the corresponding **SourceCard** in the right panel.

The model is instructed to refuse if the context is insufficient — that is the second line of defence after the CRAG grader.

## 5.2 Streaming SSE + token accounting

We stream chat completions through Server-Sent Events with this event vocabulary:

```
event: meta      - retrieval results + per-stage latency
event: stage     - per-stage start / progress / done payloads
event: token     - single token delta (just the text)
event: done      - citations, total ms, total tokens
event: error     - any failure
```

Token usage is accumulated per stage using a Python `ContextVar` so each pipeline call's spend lands in the right stage bucket. Because Python `ContextVars` do **not** propagate to executor threads, we capture a reference up-front and write into it from inside the thread:

```
acc_ref = current_acc()
def _produce():
    # ...stream code...
    _add_to(acc_ref, prompt_tokens, completion_tokens)
loop.run_in_executor(None, _produce)
```

That's what keeps the per-stage token bar accurate across the UI.

---

## 6. Deployment topology

### 6.1 Provider abstraction — Stellar vs Vertex

The whole pipeline talks to one interface (`get_stellar()` returns either a `StellarClient` or `VertexClient`):

```
class Provider(Protocol):
    async def embed(self, texts: str | list[str]) → list[list[float]]
    async def embed_one(self, text: str) → list[float]
    async def chat(self, model, messages, **kw) → (str, TokenUsage)
    async def chat_stream(self, model, messages, **kw) → AsyncIterator[str]
```

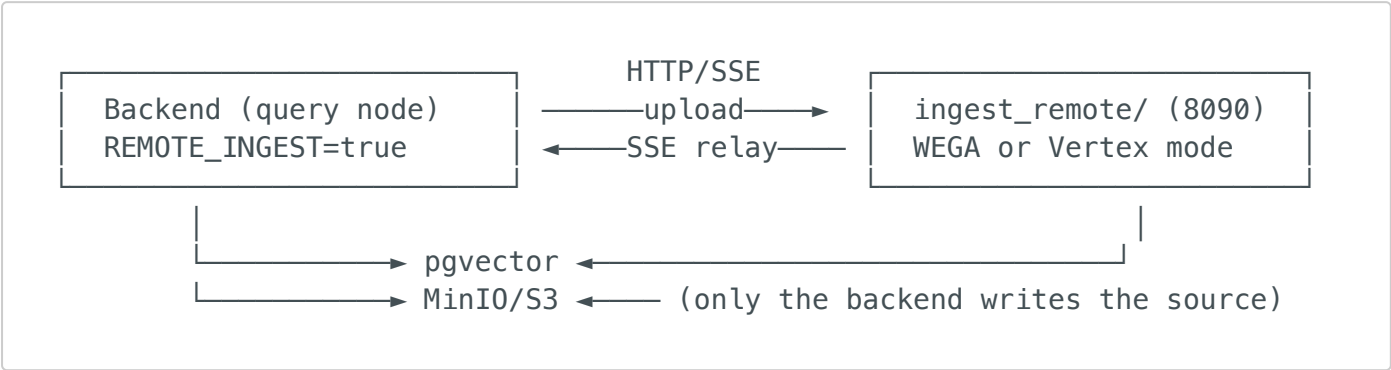
Provider selection is `LLM_PROVIDER=stellar|vertex`. Model names per task (`final_gen`, `rerank`, `fast`, `contextual`, `embedding`) are looked up via `model_for(task)` which prefers the provider-specific setting and falls back to Stellar defaults.

`StellarClient` reuses the project's `get_llm.py` (`StellarGenAI` class) which handles **COIN OAuth token refresh** by rebuilding the underlying `openai.OpenAI` client every 14 minutes — so a long-running service never hits an expired token mid-request.

`VertexClient` does the same TTL dance over a `google.genai.Client` and captures token usage from the response's `usage_metadata` so the analytics tab still works without Stellar.

## 6.2 Remote ingestion service

Production deployment splits ingestion from query serving:



Two modes inside the remote service:

| LLM_PROVIDER | Chunker              | Embeddings                             | Wheel? |
|--------------|----------------------|----------------------------------------|--------|
| wega         | WegaChunker SDK      | Stellar (via <code>get_llm.py</code> ) | yes    |
| vertex       | pypdf + char chunker | Vertex <code>text-embedding-005</code> | no     |

The dispatch lives in `ingest_remote/app/main.py` — same SSE wire format for both, so the frontend never sees the difference.

## 6.3 S3 / MinIO object store

Every uploaded PDF is persisted to S3 **before** chunking, so View / Download still work even when the actual chunker runs on a different VM:

```
function persist_source(file, filename):
  s3_key = f"docs/{uuid().hex[:8]}/{filename}"
  s3_put(s3_key, file.read(), content_type="application/pdf")
  upsert into vector.documents(name, s3_uri, size_bytes, content_type,
                              uploaded_at) values (...)
  on conflict (name)
  do update set s3_uri = excluded.s3_uri, ...
```

The `/api/documents/{name}/view` endpoint streams it back with **Content-Disposition: inline** (so an `<iframe>` previews it in-page), `/download` does the same with **attachment**.

All S3 details are env-driven via the **Settings** class: **S3\_ENABLED**, **S3\_ENDPOINT\_URL**, **S3\_REGION**, **S3\_BUCKET**, **S3\_ACCESS\_KEY**, **S3\_SECRET\_KEY**, **S3\_USE\_SSL**. Defaults point at a local MinIO; clearing **S3\_ENDPOINT\_URL** switches to real AWS.

---

## 7. End-to-end algorithmic walkthrough

---

A concrete `/api/chat` request with default flags (**hybrid=true**, **rerank=true**, **mmr=true**, **use\_contextual=true**, **top\_k=8**):

```
1. parse request
   query ← "Summarise Microsoft's cloud revenue trends from the 10-Q"
   strategy ← Strategy(hybrid=T, rerank=T, mmr=T, use_contextual=T,
                      rewrite=F, hyde=F, crag=F, top_k=8)

2. (rewrite skipped – flag off)
3. (hyde      skipped – flag off)

4. embed
   v0 ← await provider.embed_one(query)           # 768-D vector

5. dense_search(v0, top_k=50, use_contextual=True)
   → SQL: ORDER BY context_embedding <=> v0 LIMIT 50
   → 50 candidates with cosine scores

6. sparse_search(query, top_k=50)
   → SQL: websearch_to_tsquery + ts_rank_cd LIMIT 50
   → 50 candidates with lexical scores
```

```

7. fuse with RRF
rankings = {"dense_v0": dense, "sparse": sparse}
fused = reciprocal_rank_fusion(rankings, k=60, top_k=20)
→ 20 candidates with rrf_score + per-source components

8. rerank top 20 (rag_rerank_topn)
prompt = build_listwise_prompt(query, fused[:20])
order = await llm.chat(model=RERANK_MODEL, prompt=prompt, ...)
reranked = apply_permutation(fused, order)
→ same 20 candidates, fresh order (top is usually the answer)

9. MMR over reranked[:20], λ=0.7, k=8
selected = greedy_diversify_by_cosine_sim on embedding
→ final 8 hits

10. (crag      skipped – flag off)

11. emit `meta` event with hits + per-stage latency + tokens
12. build generation prompt with the 8 chunks numbered [1]...[8]
13. provider.chat_stream(model=FINAL_GEN_MODEL, prompt=...)
    for token in stream:
        emit `token` event(token)
14. on stream end: emit `done` with citations[], total_ms, token_usage

```

UI sees the pipeline tracker chips lit up one by one with their per-stage ms + tokens. The right column populates with the 8 source cards. The answer body streams in mid-flight with [N] chips substituted for the raw markers. CRAG would have inserted a confidence badge if it had been on.

## 8. Tuning cheatsheet

| Symptom                                 | First lever to pull                  |
|-----------------------------------------|--------------------------------------|
| "Question vocab doesn't match docs"     | turn <b>rewrite</b> on (n=3 default) |
| "Under-specified questions miss recall" | turn <b>hyde</b> on                  |
| "Top-1 is wrong but answer is in top-5" | rely on <b>rerank</b> (default on)   |
| "Top-K are all about the same thing"    | drop MMR λ from 0.7 to 0.5           |

| Symptom                              | First lever to pull                                                                            |
|--------------------------------------|------------------------------------------------------------------------------------------------|
| "Retrieval finds wrong doc entirely" | turn on <b>contextual chunking</b> during ingestion ( <a href="#">/api/ingest/contextual</a> ) |
| "Answer hallucinates"                | turn <b>crag</b> on; lower top_k                                                               |
| "Latency too high"                   | drop <b>rag_rerank_topn</b> 20 → 10                                                            |
| "Tokens too expensive"               | use 8B rerank model                                                                            |

Key config defaults (see [backend/app/config.py](#)):

```
rag_topk_dense      = 50      # first-stage dense candidates
rag_topk_sparse     = 50      # first-stage sparse candidates
rag_rrf_k           = 60      # smoothing constant
rag_rerank_topn     = 20      # how many to send to the LLM reranker
rag_final_k         = 8       # final number of chunks shown
rag_mmr_lambda      = 0.7     # relevance vs diversity (1=rel, 0=div)
```

---

## 9. References

### Foundational papers

- Robertson & Sparck-Jones (1995) — **BM25** scoring.
- Carbonell & Goldstein (1998) — *The Use of MMR for Reordering Documents* — SIGIR.
- Malkov & Yashunin (2016) — *Efficient and Robust ANN Search Using HNSW* — TPAMI.

### Modern RAG techniques

- Cormack, Clarke, Büttcher (2009) — *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods* — SIGIR.
- Gao et al. (2022) — *Precise Zero-Shot Dense Retrieval Without Relevance Labels* (HyDE) — [arXiv:2212.10496](#).
- Sun et al. (2023) — *Is ChatGPT Good at Search? Investigating LLMs as Re-Ranking Agents* (RankGPT) — [arXiv:2304.09542](#).

- Pradeep et al. (2024) — *RankZephyr: Effective and Robust Zero-Shot Listwise Reranking*.
- Yan et al. (2024) — *Corrective Retrieval Augmented Generation (CRAG)* — [arXiv:2401.15884](https://arxiv.org/abs/2401.15884).
- Anthropic (Sept 2024) — *Contextual Retrieval* — <https://www.anthropic.com/news/contextual-retrieval>.

## Storage / infrastructure

- pgvector — <https://github.com/pgvector/pgvector>
- Postgres FTS — <https://www.postgresql.org/docs/current/textsearch.html>
- MinIO — <https://min.io/>

---

Generated for the RAG Studio framework — [~/retrieval/](#). Authored 2026-05-14.